📘 Data Handling using Pandas & Data Visualization

Introduction to NumPy | Pandas | Series & DataFrames | Operations | Import/Export | Matplotlib | Line, Bar, Histogram Plots | Customization

Introduction to NumPy

Definition: NumPy (Numerical Python) is a fundamental library for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently.

Important Points:

- NumPy is the foundation for many other data science libraries like Pandas, Matplotlib, and Scikit-learn.
- It provides the `ndarray` object, which is a fast and space-efficient multi-dimensional array.
- NumPy arrays are homogeneous (all elements must be of the same data type), unlike Python lists.
- NumPy operations are much faster than equivalent operations on Python lists due to vectorization.
- Common import convention: `import numpy as np`

1. Creation of NumPy Arrays from List

Purpose: Converts a Python list into a NumPy array.

Syntax: `np.array(list)`

Parameters:

list (compulsory) — any Python list (can be nested for multi-dimensional arrays).

Return: NumPy ndarray

- Creates a 1D array from a simple list.
- Creates a 2D array from a list of lists.
- Data type is inferred from the list elements; can be explicitly specified using `dtype` parameter.

```
import numpy as np

# 1D array from list
list1 = [10, 20, 30, 40]
arr1 = np.array(list1)
print(arr1)           # [10 20 30 40]
print(type(arr1))     # <class 'numpy.ndarray'>

# 2D array from list of lists
list2 = [[1, 2, 3], [4, 5, 6]]
arr2 = np.array(list2)
print(arr2)
# Output:
# [[1 2 3]
#  [4 5 6]]

# Specifying data type
arr3 = np.array([1, 2, 3], dtype=float)
print(arr3)           # [1. 2. 3.]
```

2. Difference Between NumPy Array and Python List

Purpose: To understand when to use a NumPy array versus a Python list.

- **Data Type:** NumPy arrays are *homogeneous* (all elements must be of the same data type), while Python lists are *heterogeneous* (can hold elements of different data types).
- **Memory Efficiency:** NumPy arrays are more memory-efficient because they store data in contiguous memory blocks, whereas Python lists store references to objects.
- **Speed:** NumPy operations are significantly faster due to vectorization and implementation in C, while Python lists require loops for element-wise operations.
- **Functionality:** NumPy provides a wide range of mathematical and statistical functions (e.g., `mean()`, `std()`, `dot()`), while lists have limited built-in methods.
- **Size:** NumPy arrays have a fixed size once created (resizing creates a new array), while Python lists can grow or shrink dynamically.
- **Indexing:** Both support indexing and slicing, but NumPy supports advanced indexing (e.g., boolean indexing, fancy indexing) and multi-dimensional slicing.
- **Use Case:** NumPy arrays are preferred for numerical computations and large datasets; lists are preferred for general-purpose storage and when data types vary.

```
import numpy as np

# Python List
py_list = [1, 2, 3, 'hello', 4.5]  # heterogeneous
print(py_list)

# NumPy Array
np_arr = np.array([1, 2, 3, 4, 5])  # homogeneous
print(np_arr)

# Speed comparison (conceptual)
# list1 + list2 concatenates lists
# arr1 + arr2 performs element-wise addition

list1 = [1, 2, 3]
list2 = [4, 5, 6]
print(list1 + list2)  # [1, 2, 3, 4, 5, 6]

arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
print(arr1 + arr2)    # [5 7 9]
```

Real-World Use of Pandas and NumPy

Why are Pandas and NumPy important in the real world?

- **Data Analysis:** Used extensively in data science to clean, transform, and analyze large datasets.
- **Financial Analysis:** NumPy and Pandas are used for stock market analysis, risk management, and algorithmic trading.
- **Machine Learning:** NumPy arrays are the standard input format for most machine learning libraries (e.g., Scikit-learn, TensorFlow, PyTorch).
- **Scientific Research:** Used in physics, biology, and engineering for simulations, signal processing, and statistical analysis.
- **Business Intelligence:** Pandas is used to process Excel/CSV files, generate reports, and create dashboards.
- **Web Analytics:** Analyzing user behavior, clickstream data, and A/B testing results.

- **Image Processing:** NumPy arrays represent images as multi-dimensional arrays for manipulation and analysis.
- **Time Series Analysis:** Pandas provides powerful tools for working with dates, times, and time-based data.
- **E-commerce:** Recommendation systems, inventory management, and customer segmentation.
- **Healthcare:** Analyzing patient data, medical imaging, and clinical trial results.

Pandas Series

Definition: A Series is a one-dimensional labeled array capable of holding any data type (integers, strings, floats, Python objects, etc.). It is like a single column of a spreadsheet.

Why Do We Create a Series? (Purpose)

- **Labeled Data:** Series allows you to associate labels (index) with each data point, making data more meaningful and accessible.
- **Efficient Storage:** Series is more memory-efficient than Python lists for numerical data.
- **Vectorized Operations:** Mathematical operations can be performed on the entire Series at once, without writing loops.
- **Handling Missing Data:** Series provides methods to handle missing values (NaN) gracefully.
- **Integration with DataFrames:** A DataFrame is essentially a collection of Series, so understanding Series is crucial for working with DataFrames.
- **Time Series Data:** Series is ideal for representing time-series data where each data point has a timestamp.
- **Data Alignment:** When performing operations between two Series, Pandas automatically aligns data based on index labels.

Basic Syntax: `pd.Series(data=None, index=None, dtype=None, name=None, copy=False)`

- `data` : The data to be stored (list, ndarray, dict, scalar).
- `index` : Labels for the data (optional; default is 0 to n-1).
- `dtype` : Data type of the elements (optional; inferred if not specified).
- `name` : Name of the Series (optional).
- `copy` : If True, copies the data (default is False).

1. Creation of Series from ndarray

Purpose: Creates a Series from a NumPy array.

Syntax: `pd.Series(data, index=index)`

- If no index is provided, default integer index (0 to n-1) is used.
- The data type is inferred from the ndarray.

```
import pandas as pd
import numpy as np

data = np.array([10, 20, 30, 40])
s = pd.Series(data)
print(s)
# Output:
# 0    10
# 1    20
# 2    30
# 3    40
# dtype: int64
```

2. Creation of Series from Dictionary

Purpose: Creates a Series from a Python dictionary. Keys become the index.

Syntax: `pd.Series(dictionary)`

- The dictionary keys are used as the index labels.
- Values are the data points.

```
import pandas as pd

data = {'a': 10, 'b': 20, 'c': 30}
s = pd.Series(data)
print(s)
# Output:
# a    10
# b    20
# c    30
# dtype: int64
```

3. Creation of Series from Scalar Value

Purpose: Creates a Series with a single scalar value repeated for each index.

Syntax: `pd.Series(scalar_value, index=index)`

- An index must be provided to define the length of the Series.

```
import pandas as pd

s = pd.Series(5, index=['a', 'b', 'c', 'd'])
print(s)
# Output:
# a    5
# b    5
# c    5
# d    5
# dtype: int64
```

3.1 What is a Scalar Value?

Definition:

A scalar value is a single, indivisible value that represents a single quantity. It is not a collection or array.

- **Examples:** `5`, `3.14`, `'Hello'`, `True` are all scalar values.
- **Non-examples:** `[1, 2, 3]` (list), `{'a': 1}` (dictionary), `np.array([1, 2])` (array) are not scalars.
- In the context of Series creation, a scalar value is broadcasted (repeated) across all index positions.

```
import pandas as pd

# Scalar value 100 repeated across 3 index labels
s = pd.Series(100, index=['x', 'y', 'z'])
print(s)
# Output:
# x    100
# y    100
# z    100
# dtype: int64
```

```
# Another example: scalar boolean value
s2 = pd.Series(True, index=[1, 2, 3, 4])
print(s2)
# Output:
# 1    True
# 2    True
# 3    True
# 4    True
# dtype: bool
```

4. More Examples of Series Creation in Different Scenarios

Purpose: To understand various ways a Series can be created for different data scenarios.

```
import pandas as pd
import numpy as np

# Scenario 1: Series from a list of strings
fruits = pd.Series(['Apple', 'Banana', 'Cherry'])
print(fruits)
# Output:
# 0     Apple
# 1    Banana
# 2    Cherry
# dtype: object

# Scenario 2: Series with custom index from a list
marks = pd.Series([85, 92, 78], index=['Math', 'Science', 'English'])
print(marks)
# Output:
# Math       85
# Science    92
# English    78
# dtype: int64

# Scenario 3: Series from a dictionary with mixed data types
mixed = pd.Series({'Name': 'Alice', 'Age': 25, 'Score': 89.5})
print(mixed)
# Output:
# Name     Alice
# Age         25
# Score     89.5
# dtype: object
```

```
# Scenario 4: Series from a range of numbers
range_series = pd.Series(range(1, 6))
print(range_series)
# Output:
# 0    1
# 1    2
# 2    3
# 3    4
# 4    5
# dtype: int64

# Scenario 5: Series with datetime index
dates = pd.date_range('2023-01-01', periods=4)
values = pd.Series([10, 20, 30, 40], index=dates)
print(values)
# Output:
# 2023-01-01    10
# 2023-01-02    20
# 2023-01-03    30
# 2023-01-04    40
# Freq: D, dtype: int64

# Scenario 6: Empty Series
empty = pd.Series()
print(empty)
# Output: Series([], dtype: float64)

# Scenario 7: Series with duplicate index values
dup = pd.Series([1, 2, 3], index=['a', 'a', 'b'])
print(dup)
# Output:
# a    1
# a    2
# b    3
# dtype: int64
```

5. Mathematical Operations on Series

Purpose: Perform element-wise mathematical operations.

- Operations are aligned by index. If indices don't match, the result is `NaN` (Not a Number).
- Common operations: `+`, `-`, `*`, `/`, `**`.

```
import pandas as pd

s1 = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
s2 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])

print(s1 + s2)
# Output:
# a    11
# b    22
# c    33
# dtype: int64

print(s1 * 2)
# Output:
# a    20
# b    40
# c    60
# dtype: int64
```

5.1 More Mathematical Operations on Series

Purpose: To explore additional mathematical and statistical operations available for Series.

```
import pandas as pd
import numpy as np

s = pd.Series([10, 20, 30, 40, 50])

# Basic arithmetic
print(s + 5)        # Add 5 to each element
print(s - 5)        # Subtract 5 from each element
print(s * 2)        # Multiply each element by 2
print(s / 2)        # Divide each element by 2
print(s ** 2)       # Square each element

# Statistical operations
print(s.sum())      # 150
print(s.mean())     # 30.0
print(s.median())   # 30.0
print(s.mode())     # 0    10, 1    20, ... (all values)
print(s.std())      # 15.811388300841896
print(s.var())      # 250.0
```

```
print(s.min())      # 10
print(s.max())      # 50
print(s.count())    # 5

# Cumulative operations
print(s.cumsum())   # Cumulative sum
print(s.cumprod())  # Cumulative product

# Operations with mismatched indices
s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s2 = pd.Series([4, 5, 6], index=['b', 'c', 'd'])
print(s1 + s2)
# Output:
# a    NaN
# b    6.0
# c    8.0
# d    NaN
# dtype: float64

# Handling NaN: fill_value
print(s1.add(s2, fill_value=0))
# Output:
# a    1.0
# b    6.0
# c    8.0
# d    6.0
# dtype: float64
```

6. Head() and Tail() Functions

Purpose: To view the first or last few elements of a Series.

Syntax: `series.head(n)` / `series.tail(n)`

Parameters: `n` (optional) — number of elements to display (default is 5).

```
import pandas as pd

s = pd.Series([10, 20, 30, 40, 50, 60, 70])
print(s.head(3))
# Output:
# 0    10
# 1    20
# 2    30
```

```
# dtype: int64

print(s.tail(2))
# Output:
# 5    60
# 6    70
# dtype: int64
```

6.1 Head() and Tail() Without Parameters (Default 5)

Purpose:

To understand the default behavior of head() and tail() when no argument is passed.

- When no argument is passed, `head()` returns the first 5 elements.
- When no argument is passed, `tail()` returns the last 5 elements.
- If the Series has fewer than 5 elements, all elements are returned.

```
import pandas as pd

# Series with more than 5 elements
s = pd.Series([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
print("Head (default):")
print(s.head())
# Output:
# 0    10
# 1    20
# 2    30
# 3    40
# 4    50
# dtype: int64

print("Tail (default):")
print(s.tail())
# Output:
# 5    60
# 6    70
# 7    80
# 8    90
# 9    100
# dtype: int64

# Series with fewer than 5 elements
s2 = pd.Series([1, 2, 3])
```

```
print("Head (fewer than 5):")
print(s2.head())
# Output:
# 0    1
# 1    2
# 2    3
# dtype: int64

print("Tail (fewer than 5):")
print(s2.tail())
# Output:
# 0    1
# 1    2
# 2    3
# dtype: int64
```

7. Selection, Indexing and Slicing

Purpose: Accessing specific elements or subsets of a Series.

- **Label-based indexing:** Use the index label (e.g., `s['a']`).
- **Positional indexing:** Use integer position (e.g., `s[0]`).
- **Slicing:** Use `start:end` (end is excluded). Works with both labels and positions.

```
import pandas as pd

s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
print(s['b'])       # 20 (label-based)
print(s[1])         # 20 (positional)
print(s['a':'c'])   # a    10, b    20, c    30
print(s[0:2])       # a    10, b    20
```

7.1 More Examples for Selection, Slicing, Indexing

Purpose: To practice and understand various selection and indexing scenarios.

```
import pandas as pd

s = pd.Series([100, 200, 300, 400, 500], index=['a', 'b', 'c', 'd', 'e'])

# 1. Accessing a single element by label
print(s['c'])          # 300
```

```python
# 2. Accessing a single element by position
print(s[2])            # 300

# 3. Slicing with labels (inclusive of end label)
print(s['b':'d'])
# Output:
# b    200
# c    300
# d    400
# dtype: int64

# 4. Slicing with positions (exclusive of end position)
print(s[1:4])
# Output:
# b    200
# c    300
# d    400
# dtype: int64

# 5. Negative indexing
print(s[-1])           # 500 (last element)
print(s[-3:])          # c    300, d    400, e    500

# 6. Fancy indexing (using a list of labels)
print(s[['a', 'c', 'e']])
# Output:
# a    100
# c    300
# e    500
# dtype: int64

# 7. Boolean indexing
print(s[s > 250])
# Output:
# c    300
# d    400
# e    500
# dtype: int64

# 8. Using .loc (label-based)
print(s.loc['b'])      # 200
print(s.loc['b':'d'])  # b    200, c    300, d    400
```

```
# 9. Using .iloc (position-based)
print(s.iloc[1])       # 200
print(s.iloc[1:4])     # b    200, c    300, d    400

# 10. Slicing with step
print(s[::2])
# Output:
# a    100
# c    300
# e    500
# dtype: int64

# 11. Reverse the Series
print(s[::-1])
# Output:
# e    500
# d    400
# c    300
# b    200
# a    100
# dtype: int64
```

8. NaN (Not a Number)

Definition:

NaN (Not a Number) is a special floating-point value used to represent missing or undefined data in Pandas and NumPy.

- **Origin:** NaN is derived from the IEEE 754 floating-point standard. It is used when a mathematical operation is undefined (e.g., 0/0) or when data is missing.
- **Type:** NaN is a float value. When NaN is present in an integer Series, the Series is automatically converted to float.
- **Detection:** Use `pd.isna()` or `pd.isnull()` to check for NaN values. Use `pd.notna()` or `pd.notnull()` for the opposite.
- **Handling Missing Data:** Pandas provides methods like `fillna()`, `dropna()`, and `interpolate()` to handle NaN values.
- **Impact on Operations:** Any arithmetic operation involving NaN results in NaN (unless using methods with `fill_value`).
- **NaN is not equal to NaN:** In Python, `np.nan == np.nan` returns `False`. Use `pd.isna()` to check for NaN.

```
import pandas as pd
import numpy as np

# Creating a Series with NaN values
s = pd.Series([1, 2, np.nan, 4, np.nan])
print(s)
# Output:
# 0    1.0
# 1    2.0
# 2    NaN
# 3    4.0
# 4    NaN
# dtype: float64

# Detecting NaN values
print(pd.isna(s))
# Output:
# 0    False
# 1    False
# 2     True
# 3    False
# 4     True
# dtype: bool

print(pd.notna(s))
# Output:
# 0     True
# 1     True
# 2    False
# 3     True
# 4    False
# dtype: bool

# Handling NaN: fillna()
print(s.fillna(0))
# Output:
# 0    1.0
# 1    2.0
# 2    0.0
# 3    4.0
# 4    0.0
# dtype: float64

# Handling NaN: dropna()
```

```
print(s.dropna())
# Output:
# 0    1.0
# 1    2.0
# 3    4.0
# dtype: float64

# NaN in operations
s1 = pd.Series([1, 2, np.nan])
s2 = pd.Series([4, np.nan, 6])
print(s1 + s2)
# Output:
# 0    5.0
# 1    NaN
# 2    NaN
# dtype: float64

# NaN is not equal to itself
print(np.nan == np.nan)  # False
print(pd.isna(np.nan))   # True

# NaN in a DataFrame
df = pd.DataFrame({
    'A': [1, 2, np.nan],
    'B': [4, np.nan, 6]
})
print(df)
# Output:
#      A    B
# 0  1.0  4.0
# 1  2.0  NaN
# 2  NaN  6.0

print(df.dropna())  # Drops rows with any NaN
print(df.fillna(0)) # Fills NaN with 0
print(df.fillna(method='ffill')) # Forward fill
print(df.fillna(method='bfill')) # Backward fill
```

Pandas DataFrames

Definition: A DataFrame is a two-dimensional labeled data structure with columns of potentially different types. It is like a spreadsheet or SQL table.

Purpose of a DataFrame

- **Tabular Data Representation:** DataFrames represent data in a table format with rows and columns, which is intuitive and easy to understand.
- **Heterogeneous Data:** Unlike NumPy arrays, DataFrames can hold columns of different data types (e.g., integers, floats, strings).
- **Labeled Axes:** Both rows and columns have labels, making data access and manipulation more meaningful.
- **Data Alignment:** DataFrames automatically align data based on labels, reducing errors in calculations.
- **Missing Data Handling:** DataFrames provide robust methods to handle missing data (NaN).
- **Integration with CSV/Excel:** DataFrames can be easily imported from and exported to CSV, Excel, and SQL databases.
- **Powerful Operations:** Supports filtering, grouping, aggregation, merging, and reshaping of data.
- **Foundation for Data Analysis:** DataFrames are the primary data structure used in data analysis and machine learning workflows.

Why Do Index Positions Come in a DataFrame?

- **Unique Identification:** Index positions provide a unique identifier for each row, making it easy to refer to specific rows.
- **Data Alignment:** When performing operations between two DataFrames, Pandas aligns data based on index labels, ensuring correct matching of rows.
- **Efficient Access:** Indexing allows for fast retrieval of rows using labels or positions.
- **Slicing and Filtering:** Index positions are used to slice DataFrames (e.g., `df[0:5]`) and filter data.
- **Default Index:** If no index is specified, Pandas automatically assigns a default integer index (0, 1, 2, ...).
- **Custom Index:** You can set a custom index (e.g., dates, IDs) to make data more meaningful.

- **Hierarchical Indexing:** Pandas supports multi-level indexing (MultiIndex) for more complex data structures.
- **Time Series:** Index positions are crucial for time-series data, where the index represents timestamps.

```
import pandas as pd

# Default index (0, 1, 2, ...)
df = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Age': [25, 30]})
print(df)
# Output:
#     Name  Age
# 0  Alice   25
# 1    Bob   30

# Custom index
df2 = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Age': [25, 30]}, index=['a',
'b'])
print(df2)
# Output:
#     Name  Age
# a  Alice   25
# b    Bob   30
```

0. Series vs DataFrame — Comparison Table

Purpose: To clearly understand the differences between Series and DataFrame.

Feature	Series	DataFrame
Dimensionality	1D	2D
Structure	Single column	Multiple columns (table)
Index	Single index	Row index + Column index
Data Types	Homogeneous (generally)	Heterogeneous (columns can differ)
Creation	`pd.Series()`	`pd.DataFrame()`
Example	`pd.Series([1,2,3])`	`pd.DataFrame({'A':[1,2]})`
Use Case	Single variable data	Multiple variables data

1. Creation from Dictionary of Series

Purpose:

Creates a DataFrame from a dictionary where keys are column names and values are Series.

```
import pandas as pd

d = {
    'Name': pd.Series(['Alice', 'Bob', 'Charlie']),
    'Age': pd.Series([25, 30, 35]),
    'City': pd.Series(['NY', 'LA', 'Chicago'])
}
df = pd.DataFrame(d)
print(df)
# Output:
#       Name  Age     City
# 0    Alice   25       NY
# 1      Bob   30       LA
# 2  Charlie   35  Chicago
```

2. Creation from List of Dictionaries

Purpose: Creates a DataFrame from a list where each dictionary represents a row.

```
import pandas as pd

data = [
    {'Name': 'Alice', 'Age': 25},
    {'Name': 'Bob', 'Age': 30},
    {'Name': 'Charlie', 'Age': 35}
]
df = pd.DataFrame(data)
print(df)
# Output:
#       Name  Age
# 0    Alice   25
# 1      Bob   30
# 2  Charlie   35
```

3. Creation from Text/CSV Files

Purpose: Reads data from a CSV (Comma Separated Values) file into a DataFrame.

Syntax: `pd.read_csv('filename.csv')`

- The first row is assumed to be the header (column names) by default.
- Use `header=None` if the file has no header row.
- Use `sep` parameter to specify a different delimiter (default is comma).

```
import pandas as pd

# Assuming 'data.csv' contains:
# Name,Age,City
# Alice,25,NY
# Bob,30,LA
df = pd.read_csv('data.csv')
print(df)

# Using sep parameter for different delimiters
df = pd.read_csv('data.csv', sep=',')      # comma (default)
df = pd.read_csv('data.txt', sep='\t')     # tab-separated
df = pd.read_csv('data.csv', sep=';')      # semicolon-separated
df = pd.read_csv('data.csv', sep='|')      # pipe-separated
```

3.1 Creating an Empty DataFrame

Purpose: Creates a DataFrame with no rows and no columns.

Syntax: `pd.DataFrame()`

- Returns an empty DataFrame with 0 rows and 0 columns.
- Useful as a starting point when building a DataFrame dynamically.

```
import pandas as pd

# Creating an empty DataFrame
empty_df = pd.DataFrame()
print(empty_df)
# Output:
# Empty DataFrame
# Columns: []
# Index: []
```

```
print(empty_df.shape)  # (0, 0)

# Adding columns to an empty DataFrame
empty_df['Name'] = ['Alice', 'Bob']
empty_df['Age'] = [25, 30]
print(empty_df)
# Output:
#     Name  Age
# 0  Alice   25
# 1    Bob   30
```

4. Display, Iteration, Head and Tail

Purpose: To view and iterate over DataFrame data.

- `df.head(n)` : First n rows (default 5).
- `df.tail(n)` : Last n rows (default 5).
- `df.columns` : Get column names.
- `df.index` : Get row index.
- **Iteration:** `for index, row in df.iterrows():` (iterates over rows).

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35]}
df = pd.DataFrame(data)

print(df.head(2))
print(df.columns)  # Index(['Name', 'Age'], dtype='object')

for index, row in df.iterrows():
    print(index, row['Name'], row['Age'])

# Default head() and tail()
df2 = pd.DataFrame({'A': range(1, 11)})
print(df2.head())   # First 5 rows
print(df2.tail())   # Last 5 rows
print(df2.tail(3))  # Last 3 rows
```

5. Operations on Rows and Columns: Add, Select, Delete, Rename

Purpose: Modifying the structure of a DataFrame.

- **Add Column:** `df['NewColumn'] = [values]`
- **Select Column:** `df['ColumnName']` or `df.ColumnName`
- **Delete Column:** `del df['ColumnName']` or `df.drop('ColumnName', axis=1, inplace=True)`
- **Rename Column:** `df.rename(columns={'OldName': 'NewName'}, inplace=True)`
- **Add Row:** `df.loc[new_index] = [values]`
- **Delete Row:** `df.drop(index_label, inplace=True)`

```
import pandas as pd

data = {'Name': ['Alice', 'Bob'], 'Age': [25, 30]}
df = pd.DataFrame(data)

# Add a new column
df['City'] = ['NY', 'LA']

# Select a column
print(df['Name'])

# Rename a column
df.rename(columns={'Name': 'FirstName'}, inplace=True)

# Delete a column
del df['City']
print(df)

# Add a new row using loc
df.loc[2] = ['Charlie', 35]
print(df)

# Delete a row by index label
df.drop(index=1, inplace=True)
print(df)

# Drop multiple rows
df.drop(index=[0, 2], inplace=True)
print(df)

# Drop a column using drop()
```

```
df.drop('FirstName', axis=1, inplace=True)
print(df)
```

6. Indexing using Labels and Boolean Indexing

Purpose: Accessing specific data points or filtering data.

- **Label Indexing:** Use `df.loc[row_label, column_label]`.
- **Positional Indexing:** Use `df.iloc[row_position, column_position]`.
- **Boolean Indexing:** Use a condition to filter rows, e.g., `df[df['Age'] > 25]`.

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35]}
df = pd.DataFrame(data)

# Label-based access
print(df.loc[1, 'Name'])  # Bob

# Positional access
print(df.iloc[2, 1])      # 35

# Boolean Indexing
filtered_df = df[df['Age'] > 28]
print(filtered_df)

# df.loc[:] - selects ALL rows and columns
print(df.loc[:])   # Displays entire DataFrame

# df.loc[0:2] - label-based slicing (INCLUSIVE of end label)
print(df.loc[0:2]) # Displays rows with labels 0, 1, 2 (3 rows)

# df.iloc[0:2] - positional slicing (EXCLUSIVE of end)
print(df.iloc[0:2]) # Displays rows at positions 0, 1 (2 rows)
```

6.1 df.loc[] vs df.iloc[] — Comparison Table

Purpose: To clearly understand the difference between label-based and positional indexing.

Feature	`df.loc[]`	`df.iloc[]`
Based on	Labels	Integer positions

Feature	`df.loc[]`	`df.iloc[]`
Slicing	Inclusive of end	Exclusive of end
Example	`df.loc['a':'c']`	`df.iloc[0:3]`
Error	KeyError if label missing	IndexError if position out of range
Use Case	When index labels are meaningful	When position matters

```
import pandas as pd

df = pd.DataFrame({'A': [10, 20, 30, 40]}, index=['a', 'b', 'c', 'd'])

# loc – label-based (inclusive)
print(df.loc['a':'c'])
# Output:
#     A
# a  10
# b  20
# c  30

# iloc – position-based (exclusive)
print(df.iloc[0:3])
# Output:
#     A
# a  10
# b  20
# c  30

# loc with integer labels
df2 = pd.DataFrame({'A': [10, 20, 30, 40]})
print(df2.loc[0:2])
# Output:
#     A
# 0  10
# 1  20
# 2  30

# iloc with integer positions
print(df2.iloc[0:2])
# Output:
#     A
```

```
# 0  10
# 1  20
```

6.2 More Examples for All DataFrame Topics

Purpose: To practice and understand various DataFrame operations in different scenarios.

```
import pandas as pd

# Scenario 1: Creating a DataFrame with custom index
df = pd.DataFrame({
    'Product': ['Laptop', 'Phone', 'Tablet'],
    'Price': [1000, 500, 300],
    'Quantity': [5, 10, 8]
}, index=['P1', 'P2', 'P3'])
print(df)
# Output:
#       Product  Price  Quantity
# P1    Laptop   1000         5
# P2     Phone    500        10
# P3    Tablet    300         8

# Scenario 2: Adding a new column based on existing data
df['Total'] = df['Price'] * df['Quantity']
print(df)
# Output:
#       Product  Price  Quantity  Total
# P1    Laptop   1000         5   5000
# P2     Phone    500        10   5000
# P3    Tablet    300         8   2400

# Scenario 3: Selecting multiple columns
print(df[['Product', 'Total']])
# Output:
#       Product  Total
# P1    Laptop   5000
# P2     Phone   5000
# P3    Tablet   2400

# Scenario 4: Filtering rows using boolean indexing
print(df[df['Price'] > 400])
# Output:
#       Product  Price  Quantity  Total
```

```
# P1    Laptop   1000         5   5000
# P2     Phone    500        10   5000

# Scenario 5: Renaming multiple columns
df.rename(columns={'Product': 'Item', 'Price': 'Cost'}, inplace=True)
print(df.columns)  # Index(['Item', 'Cost', 'Quantity', 'Total'],
dtype='object')

# Scenario 6: Deleting a row by index label
df.drop('P2', inplace=True)
print(df)
# Output:
#        Item  Cost  Quantity  Total
# P1    Laptop  1000         5   5000
# P3    Tablet   300         8   2400

# Scenario 7: Adding a new row using loc
df.loc['P4'] = ['Monitor', 200, 15, 3000]
print(df)
# Output:
#         Item  Cost  Quantity  Total
# P1    Laptop  1000         5   5000
# P3    Tablet   300         8   2400
# P4   Monitor   200        15   3000

# Scenario 8: Iterating over columns
for col in df.columns:
    print(col, "->", df[col].tolist())

# Scenario 9: Using iloc for positional access
print(df.iloc[0, 0])  # Laptop (first row, first column)
print(df.iloc[1:3, 1:3])
# Output:
#     Cost  Quantity
# P3   300         8
# P4   200        15

# Scenario 10: Checking for missing values
df2 = pd.DataFrame({'A': [1, 2, None], 'B': [4, None, 6]})
print(df2.isnull())
# Output:
#        A      B
# 0  False  False
# 1  False   True
```

```
# 2   True  False

print(df2.dropna())  # Drops rows with any NaN
print(df2.fillna(0)) # Fills NaN with 0

# Scenario 11: Dropping rows by index label
df3 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
df3.drop(index=1, inplace=True)
print(df3)
# Output:
#    A  B
# 0  1  4
# 2  3  6

# Scenario 12: Dropping multiple rows
df4 = pd.DataFrame({'A': [1, 2, 3, 4], 'B': [5, 6, 7, 8]})
df4.drop(index=[0, 2], inplace=True)
print(df4)
# Output:
#    A  B
# 1  2  6
# 3  4  8
```

7. Importing/Exporting Data between CSV and DataFrames

Purpose: Reading from and writing to CSV files.

- **Import:** `pd.read_csv('filename.csv')`
- **Export:** `df.to_csv('filename.csv', index=False)` (index=False prevents writing row indices).
- **Default behavior:** `to_csv()` writes the index column as well. Use `index=False` to exclude it.

```
import pandas as pd

# Export to CSV - Default behavior (includes index)
data = {'Name': ['Alice', 'Bob'], 'Age': [25, 30]}
df = pd.DataFrame(data)
df.to_csv('output_with_index.csv')

# Export to CSV - Recommended (excludes index)
df.to_csv('output.csv', index=False)
```

```
# Import from CSV
df_new = pd.read_csv('output.csv')
print(df_new)

# Import with different delimiter
df_tab = pd.read_csv('data.txt', sep='\t')
df_semi = pd.read_csv('data.csv', sep=';')
```

Data Visualization with Matplotlib

Purpose of Plotting: To visually represent data, making it easier to identify patterns, trends, and outliers.

1. Line Plot

Purpose:

To display information as a series of data points connected by straight line segments.

Syntax: `plt.plot(x, y)`

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

plt.plot(x, y)
plt.show()
```

2. Bar Graph

Purpose: To compare categorical data using rectangular bars.

Syntax: `plt.bar(x, height)`

```
import matplotlib.pyplot as plt

categories = ['A', 'B', 'C']
values = [10, 20, 15]
```

```
plt.bar(categories, values)
plt.show()
```

3. Histogram

Purpose: To show the frequency distribution of a continuous variable.

Syntax: `plt.hist(data, bins)`

- `bins` can be an integer (number of bins) or a list (bin edges).
- Each bar's height represents the frequency of data points in that bin.

```
import matplotlib.pyplot as plt
import numpy as np

data = np.random.randn(1000)

# bins as integer (number of bins)
plt.hist(data, bins=30)
plt.show()

# bins as a list (custom bin edges)
ages = [22, 25, 24, 28, 30, 24, 29, 27, 21, 23]
bins = [20, 22, 24, 26, 28, 30]
plt.hist(ages, bins=bins, edgecolor='black')
plt.show()

# Histogram with title and labels
plt.hist(ages, bins=bins, edgecolor='black', label='Participants')
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Number of Participants')
plt.legend()
plt.show()
```

4. Customizing Plots: Label, Title, Legend

Purpose: To make plots more informative and readable.

- `plt.title('Title')` : Adds a title.
- `plt.xlabel('X-axis Label')` : Adds a label to the x-axis.
- `plt.ylabel('Y-axis Label')` : Adds a label to the y-axis.

- `plt.legend(['Label1', 'Label2'])`: Adds a legend for multiple plots.
- `plt.savefig('plot.png')`: Saves the plot to a file.
- **Important:** Call `savefig()` *before* `show()`, otherwise the saved file may be empty.

```
import matplotlib.pyplot as plt

x = [1, 2, 3]
y1 = [10, 20, 30]
y2 = [5, 15, 25]

plt.plot(x, y1, label='Line 1')
plt.plot(x, y2, label='Line 2')

plt.title('My Custom Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()

# Save the plot before show()
plt.savefig('my_plot.png')
plt.savefig('my_plot.pdf')           # Save as PDF
plt.savefig('my_plot.png', dpi=300)  # Higher resolution

plt.show()
```

5. Complete Visualization Examples

Purpose: To see complete, exam-ready examples for line plot, bar graph, and histogram.

```
import matplotlib.pyplot as plt

# ========== LINE PLOT ==========
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May']
revenue = [50, 65, 75, 80, 95]

plt.plot(months, revenue, label='Revenue')
plt.title('Monthly Revenue')
plt.xlabel('Months')
plt.ylabel('Revenue (in thousands)')
plt.legend()
plt.savefig('monthly_revenue.png')
plt.show()
```

```
# ========== BAR GRAPH ==========
products = ['Pen', 'Book', 'Pencil', 'Eraser', 'Scale']
sales = [120, 200, 150, 90, 110]

plt.bar(products, sales, label='Sales Data')
plt.title('Product Sales Report')
plt.xlabel('Products')
plt.ylabel('Sales')
plt.legend()
plt.savefig('sales.png')
plt.show()

# ========== HISTOGRAM ==========
ages = [22, 25, 24, 28, 30, 24, 29, 27, 21, 23]
bins = [20, 22, 24, 26, 28, 30]

plt.hist(ages, bins=bins, edgecolor='black', label='Participants')
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Number of Participants')
plt.legend()
plt.show()
```

Quick Reference Table

Library	Function / Method	Purpose	Returns
`numpy`	`np.array()`	Create a NumPy array	ndarray
	`np.arange()`	Create array with range	ndarray
`pandas`	`pd.Series()`	Create a 1D Series	Series
	`pd.DataFrame()`	Create a 2D DataFrame	DataFrame
	`pd.read_csv()`	Read CSV into DataFrame	DataFrame
	`df.head(n)`	First n rows	DataFrame
	`df.tail(n)`	Last n rows	DataFrame
	`df.loc[]`	Label-based indexing	DataFrame/Series

Library	Function / Method	Purpose	Returns
	`df.iloc[]`	Position-based indexing	DataFrame/Series
	`df.to_csv()`	Write DataFrame to CSV	None
`matplotlib`	`plt.plot()`	Draw a line plot	None (displays)
	`plt.bar()`	Draw a bar graph	None (displays)
	`plt.hist()`	Draw a histogram	None (displays)
	`plt.title()`	Add a title	None
	`plt.xlabel()`	Add x-axis label	None
	`plt.ylabel()`	Add y-axis label	None
	`plt.legend()`	Add a legend	None

Important Board Exam Tips Must Know

1. **NumPy Basics:**
   - NumPy is the foundation for numerical computing in Python.
   - Create arrays from lists: `np.array([1, 2, 3])`
   - NumPy arrays are faster and more memory-efficient than lists.
   - NumPy arrays are homogeneous; lists are heterogeneous.
2. **Pandas Series vs DataFrame:**
   - **Series:** 1D labeled array.
   - **DataFrame:** 2D labeled data structure (like a table).
3. **Creating DataFrames:**
   - From dictionary of Series, list of dictionaries, or CSV files.
   - Use `pd.DataFrame()` for manual creation.
   - `pd.DataFrame()` with no arguments creates an empty DataFrame.
4. **Head vs Tail:**
   - `head(n)` : First n rows (default 5).
   - `tail(n)` : Last n rows (default 5).
   - If fewer than n rows exist, all rows are returned.

5. **Indexing in DataFrame:**
   - `df.loc[]` : Label-based indexing (inclusive of end label).
   - `df.iloc[]` : Integer position-based indexing (exclusive of end position).
   - `df[condition]` : Boolean indexing for filtering.
   - `df.loc[:]` : Selects all rows and columns.
6. **Operations on Columns:**
   - Add: `df['New'] = values`
   - Delete: `del df['Column']` or `df.drop('Column', axis=1)`
   - Rename: `df.rename(columns={'Old': 'New'})`
7. **Operations on Rows:**
   - Add: `df.loc[new_index] = [values]`
   - Delete: `df.drop(index=label)` or `df.drop([labels])`
8. **CSV Import/Export:**
   - Import: `pd.read_csv('file.csv')`
   - Export: `df.to_csv('file.csv', index=False)`
   - Use `sep` parameter for different delimiters.
   - Default `to_csv()` includes index; use `index=False` to exclude.
9. **NaN Handling:**
   - NaN represents missing data.
   - Detect: `pd.isna()` , `pd.isnull()`
   - Handle: `fillna()` , `dropna()` , `interpolate()`
   - NaN is not equal to itself: `np.nan == np.nan` is `False`
10. **Matplotlib Plots:**
    - **Line Plot:** `plt.plot(x, y)` — shows trends.
    - **Bar Graph:** `plt.bar(x, height)` — compares categories.
    - **Histogram:** `plt.hist(data, bins)` — shows frequency distribution.
    - `bins` can be an integer or a list of bin edges.
11. **Customizing Plots:**
    - Add title: `plt.title()`
    - Add labels: `plt.xlabel()` , `plt.ylabel()`
    - Add legend: `plt.legend()`
    - Save plot: `plt.savefig('filename.png')` — before `show()`

12. **Common Errors to Avoid:**
    - Forgetting to import the library: `import pandas as pd`
    - Not using `index=False` when exporting to CSV (adds unwanted index column).
    - Mixing up `loc` and `iloc`.
    - Forgetting to call `plt.show()` to display the plot.
    - Calling `savefig()` after `show()` (file may be empty).
    - Forgetting that `loc` slicing is inclusive, while `iloc` slicing is exclusive.
13. **Board Exam Favourites:**
    - Write a program to create a DataFrame and display its first 5 rows.
    - Write a program to plot a bar graph for given data and add a title and labels.
    - Differentiate between `Series` and `DataFrame`.
    - Explain how to import data from a CSV file into a DataFrame.
    - Write a program to create a NumPy array from a list and perform basic operations.
    - Differentiate between NumPy array and Python list.
    - Explain the concept of NaN and how to handle it in Pandas.
    - Differentiate between `loc` and `iloc`.
    - Write a program to plot a histogram with custom bins.
